01 시스템 기본 학습

- 윈도우 인증 구성 요소 - LSA(Local Security Authority),SAM(Security Account Manager), SRM(Security Reference Monitor)
- LSA : 모든 계정의 로그인에 대한 검증 및 시스템 자원에 대한 접근 권한을 검사
- SAM : 데이터 베이스 관리 -> SID 정보도 저장 되어 있음
- SRM : SID 부여, 감사메시지 생성

- SID -> 500(관리자), 501(게스트), 1000(일반 사용자 식별자)
- WMIC(Windows Management Instruction Cosole) -> 윈도우 관리 명령 콘솔
- 실행 > wmic > useraccount list brief 를 통해 SID 조회 가능

- 윈도우 인증 방식 -> Challenge & Response
- 윈도우 인증 알고리즘 -> LM (사용 x), LTLM (LM + MD4 해시 추가), NTLMv2해시 (현재 사용하는 윈도우 시스템의 기본 인정 프로토콜)

[패스워드 크래킹]
- 사전 공격/사전 대입 공격 (Dictionary Attack)
  -> 패스워드로 자주 사용되는 사전에 있는 단어, 키보드 자판의 일련순을 미리 사전파일로 생성 후 하나씩 대입하여 패스워드 일치 여부를 확인함

- 무차별 공격/무작위 대입 공격 (Brute Force Attack)
-> 패스워드에 사용될 수 있는 문자열 범위를 지정하고 범위 내에서 생성가능한 모든패스워드를 생성하여 이를 하나씩 대입, 패스워드 일치 여부를 확인하는 패스워드 크래킹 기법 (일반적으로 사전 대입 공격 이후 무차별 공격을 수행)

- 혼합 공격 (Hybrid Attack)

-> 사전 공격 + 무차별 공격/무작위 대입 공격

- 레인보우 테이블 공격

-> 해시테이블과 함수의 반복 수행을 통해 일치하는 해시값을 통해서 패스워드를 찾아내는 방식
-> 레인보우 테이블 : 일정 수의 패스워드와 해시로 이루어진 체인을 무수히 만들어 놓은 테이블

* /etc/passwd - 리눅스에서 home 디렉터리, 로그인쉘, 사용자의 사용자 이름 (user 또는 logname) 을 지정하는 파일

-> 구분자(:) 를 이용해 7개의 필드로 구분함

user_account:user_password:user_id:group_id:comment:home_directory:login_shell

-> 해킹시 주로 user_id, group_id 를 0으로 변경해서 root 권한 탈취

* logout / exit / ctrl + d 를 통해 사용자 로그아웃 가능

* UID, GID 는 사용자와 그룹을 식별하는 식별자로 시스템에 유일한 정수값으로 기술

* /etc/group - 현재 시스템에 정의된 모든 그룹의 정보 정의

group_name:unvariable:group_id:user_account_entry_in_group

-> 총 네개의 필드로 구성되어 있음. 그룹의 암호화된 패스워드는 사용하지 않으며 보통 x 로 기재함

예시)

bin:x:1:root,bin,daemon

* 입출력 재지정 - 키보드에서 들어오는 입력을 파일에서 받도록 대체하는 것 / 명령의 실행 결과나 에러 상황을 화면에 출력하지 않고 파일로 받는 것

-입력 재지정 : command [0]< file_name
-출력 재지정 : command [1 or 2]>file_name
=> 1 은 표준 출력, 2는 표준 에러. default 는 표준 출력

표준 출력 재지정 예시)

```
who > who.txt
who >> who.txt
```

* 특수 문자(metacharacter)
-> ? 는 한문자를 의미 , * 는 글자수 제한이 없음

* 파일 시스템
: 물리적인 저장장치에 파일을 생성, 저장, 관리하기 위한 논리적인 자료구조를 파일 시스템이라고 함

* 물리적인 디스크는 논리적인 파티션으로 구분 되며 파티션 별로 고유한 파일 시스템을 생성함

* 파일 시스템 구성
  1) 부트 블럭 : 운영체제를 부팅하거나 초기화하기 위한 부트스트랩 코드를 담고 있는 블럭
  2) 슈퍼 블럭 : 파일 시스템을 관리하기 위한 정보를 담고 있음
  3) 아이노드 리스트 : 파일들에 대한 속성 정보를 담고 있음 (inode 에는 파일명이 없음 ! 파일명은 디렉터리를 통해 관리 되는 것이지 inodelist 와는 관계 없음), stat 명령어를 사용하면 아이노드 속성 정보를 확인 가능함
  4) 데이터 블럭 : 실제 파일의 내용이 저장되는 블럭으로 고정 크기로 구성

* 아이노드 구성 정보 -> inode number, 파일 타입 (일반파일/디렉터리/장치파일), 접근권한, linkcount(하드링크개수),소유자,소유그룹,파일크기,MAC Time(modification time -> 파일 내용 변경 /access time/change time -> 파일 속성 변경), block index

* 파일 시스템과 링크 파일
-> 하드링크 / 심볼릭 링크로 구분
-> ln -s : 심볼릭 링크(file, directory) / ln : 하드링크 (only file)

* 하드링크 : 기존 파일과 동일한 inode number 을 가지는 파일을 생성
* 심볼릭 링크 : 파일 경로를 기반으로 하므로 파일 시스템에 제한이 없으며 디렉터리도 가능

ln -s abc.dat abc_sl.dat 이라고 하면 abc_sl.dat 이 심볼릭 링크에 해당함

* 파일의 종류 - 일반 파일(표준 파일 입출력 시스템 호출을 통해 참조), 디렉터리 (디렉터리의 명시적인 시스템 호출을 통해 참조), 특수파일 (표준 입출력 시스템 호출을 통해 참조)

* IPC (InterProcess Communication) : 프로세스간 통신

* 장치 파일 : 블럭 단위로 입출력 (버퍼링) / 문자 단위로 입출력 (비버퍼링)

* ls : 디렉터리 내용 출력
-> ls -l : 목록형태로 출력

-> ls -a : 도트 파일을 포함하여 디렉터리 내에 있는 모든 디렉터리 및 파일을 보여줌 (숨김포함)
-> ls -R : 하위 디렉터리에 있는 내용까지 출력
-> ls -F : /, *, @ (순서대로 디렉터리, 실행파일, 심볼릭링크)등 파일 종류 확인
-> ls -i : 디렉터리 및 파일에 저장된 inode-number 확인

* 파일 종류
-> d : 디렉터리, b: 블록 장치파일, c : 문자 장치 파일, l : 심볼릭 링크파일, p : 네임드 파이프, s : 유닉스 도메인 소켓, - : 일반 정규 파일

* 파일 접근 권한
  - 파일 접근 권한 기본값 : 666, 디렉터리 : 777
  - umask : 파일 생성시 제거해야 할 접근 권한 명시

  cf : rwx 중 x 가 있어야 cd 가능 (디렉터리 안으로 이동)

* chgrp : 소유 그룹 변경

* mkdir : 디렉터리를 새로 만들 때 사용, -m 을 사용하면 명시적인 접근권한을 선언할 수 있고, -p 를 사용하면 계정 구조를 한번에 생성할 수 있다.

예시) mkdir -m 777 -p work/project

* rmdir : 디렉터리 삭제, -p 를 사용하면 명령어에 나열한 하위 디렉터리까지 한꺼번에 삭제할 수 있음.

예시) rmdir -p wokr/project

* chmod / chown / chgrp 는 기본적으로 root 권한을 가져야 실행 가능

* umask : /etc/profile 에 umask 를 지정하여 시스템 전체 사용자에게 획일적인 umask 값을 적용할 수 있음 . 일반적으로 022 많이 사용

예시) umask 022 (접근 권한을 명시하지 않으면 기존에 설정된 umask 를 출력함)

* find

- perm : 파일의 접근 권한을 지정함 (8진수로 명시), -를 안붙이면 mode 와 정확히 일치하고 -가 있으면 mode 권한을 포함하는 파일을 검색

예시) find -perm 4000 이면 setuid 만 설정된 파일 검색, find -perm -4000 이면 setuid 를 포함하는 파일 검색

- atime / ctime / mtime : += 가 없으면 정확히 n 일, +n 이면 n일 초과, -n 이면 n일 미만을 의미

예시) find -mtime -4

- type : 파일의 종류 지정

- or 은 -o, and 는 -a 를 사용하여 논리 연산을 추가할 수 있음

- exec 옵션을 통해 검색 결과 파일에 대한 추가적인 명령어를 실행할 수 있음

예시) find . \(-perm -4000 -o -perm -2000 \) -exec ls -al {} \;

- size : 단위는 b(blocks, default 의미) , c(bytes), k (kilobytes) , M (megabytes),G (gigabytes) , + 는 초과, - 는 미만

** 교재 p 26 기타 디렉터리 및 파일 관련 명령어 요약되어 있음

* PCB(Process Control Block) : 프로세스 제어 블록
 - 프로세스 상태 : ready, running, blocked
 - 프로세스 번호 : 프로세스 색별 번호
 - 프로그램 카운터 : 문맥교환 발생시 다음에 실행할 명령어의 위치값 저장
 - 레지스터 : 문맥교환이 발생할 경우 현재 프로세스의 실행 상태 정보 저장
 - 메모리 정보 : 프로세스가 사용 하는 메모리 page 또는 segment 테이블 정보

* FDT(File Discriptor Table) : 프로세스 생성시 개별 프로세스별로 오픈한 파일을 관리하기 위한 파일 디스크립터 테이블 생성, 프로세스 내에서 오픈한 각각의 파일을 식별하기 위한 양의 정수값을 FD 라고 함

* System open-file tables : 커널/운영체제가 시스템 내에서 여러 프로세스에 의해 오픈된 파일을 관리하기 위한 자료 구조

* 프로세스 특징

- 시스템 부팅을 담당하는 boot 프로세스를 제외한 모든 프로세스는 부모 프로세스를 가진다.
- 자식 프로세스가 살아 있는 상태에서 부모 프로세스가 종료하게 되면 자식 프로세스는 고아 프로세스가 됨. 이 경우 대리모 프로세스 (init proccess -> pid :1) 가 부모 역할을 한다

- 프로세스 종료시에는 자신의 종료 상태 정보(pid, exit code, cpu time) 를 부모 프로세스에 반환해야 정상적으로 소멸할 수 있다.

- 좀비 프로세스 : 프로세스 수행을 종료 했지만 부모 프로세스가 종료 상태정보를 확인하지 않아 소멸하지 않고 남아 있는 상태의 프로세스를 좀비 프로세스라고 한다
  -> 좀비 프로세스가 많아지면 더 이상 프로세스 생성이 불가능 하다
  -> Z, defunct 로 표기
  -> kill 시그널을 보내도 소멸되지 않음. 시스템 리부팅을 통해 커널 정보를 초기화 해야함

* ps - l
  - F : 프로세스 플래그 (1 : fork() 를 통해 생성되며 exec() 되지 않음, 4 : 슈퍼 유저 권한으로 실행)

  - S : 프로세스의 현재 상태 (R : 프로세스 실행중, S : 인터럽트 가능한 sleep, D : 인터럽트가 불가능한 sleep 상태. 일반적으로 I/O, T : 프로세스 정지, Z: 좀비)

  - PRI : 프로세스의 우선순위로 낮을수록 우선순위가 높음
  - NI : 프로세스의 우선순위를 계산할 때 사용하는 값
  - ADDR : 프로세스의 메모리 주소
- SZ : 프로세스가 차지하는 메모리 크기
- WCHAN : Sleeping 상태의 프로세스가 대기하는 커널 함수 명

* 프로세스 실행 순서
  - fork() : 자식 프로세스 생성 -> exec() 프로세스 이미지 전이

- 프로세스 생성과 동시에 프로세스 그룹 생성
- 터미널 제어권 : 터미널로부터 입력되는 데이터와 터미널로부터 발생하는 시그널에 대한 제어권, 프로세스 그룹 단위로 부여
- pgid 부여 / pgid 는 별도로 부여하는 식별자는 아니고 해당 그룹의 리더 프로세스 (쉘로 부터 실행된 프로세스) 의 pid 를 pgid 로 지정

* 포그라운드(foreground mode), 백그라운드(background mode) 모드
- 포그라운드 : 터미널에 대한 제어권을 가지고 동작
- 백그라운드 : 터미널에 대한 제어권 없이 동작 (&)

=> 터미널과 연결되어 세션이 생성되면 해당 세션 내에서는 하나의 포그라운드 프로세스 그룹과 하나 이상의 백그라운드 프로세스 그룹이 생성 됨

* 프로세스 관련 식별자
- PID, PPID, PGID
- SID : 세션을 식별하기 위한 ID, 터미널과 논리적인 연결 상태를 세션이라고 하며 세션 내에는 다수의 프로세스 그룹과 프로세스 그룹에 속하는 프로세스들이 존재함. SID 는 해당세션의 리더 프로세스 PID 가 됨

* ps -ef : 모든 프로세스를 보여주고 다양한 정보를 한줄씩 출력함

- STIME : 프로세스가 시작된 시간
- TTY : 프로세스와 연결된 터미널 타입 , ? 는 연결되어 있지 않음을 의미
- TIME : CPU 사용 시간
- CMD : 프로세스명

* 시그널 : UNINX 에서 지원하는 소프트웨어 인터럽트 , 다른 프로세스에 이벤트를 전달하며 전달된 이벤트를 통해 프로세스간 통신수단으로 사용함

-> 시그널은 외부, 에러, 이벤트, 인위적으로 발생함

* IPC (프로세스간 통신) 예시 : 파이프, 시그널, 메시지큐, 공유 메모리, 세마포어

* 시그널 종류
1) SIGINT (2) : ctrl + c 를 사용해 포그라운드 프로세스 그룹 내 모든 프로세스에 전달함. 디폴트로 종료

2) SIGKILL (9) : 무시하거나 임의로 처리할 수 없는 시그널. 이 시그널을 받은 프로세스는 종료

3) SIGSTOP (23) : 프로세스를 정지시키는 프로세스 제어 시그널이며 무시하거나 임의로 처리할 수 없음

4) SIGTERM (15) : 이 시그널을 받은 프로세스는 디폴트로 종료 , 시그널 옵션이 생략되면 발생

5) SIGFPE (8): 커널에 의해 발생한 산술연산 에러를 내는 프로세스에게 전달

6) SIGCHLD (18) : 프로세스의 종료나 정지의 경우에 부모 프로세스에게 이 시그널이 전달

7) SIGALRM (14) : alarm() 함수가 설정된 타이머에 의해 발생

8) SIGQUIT (3) : ctrl + \ 조합키를 누르면 전위 프로세스 그룹 내 모든 프로세스에게 이 시그널이 전달 됨. 이 시그널을 받은 프로세스는 디폴트로 종료 되며 core 파일을 생성함

9) SIGSEGV (11) : 커널에 의해 발생하며 잘못된 메모리를 참조하는 프로세스에게 이 시그널이 전달

10) SIGHUP (1) : 터미널 연결이 끊어지는 경우 프로세스에게 이 시그널이 전달 됨. 최근에는 환경설정 파일의 재설정에 사용 됨

=> 9, 23 을 받은 프로세스는 무조건 종료되거나 정지

예시) kill -9 3000 또는 kill -KILL 3000

* 런 레벨 :유닉스/리눅스 시스템 관리자가 시스템 관리의 편의성을 증대하기 위해서 사용되는 것으로 필요시에는 유닉스 부팅을 변경하여 네트워크를 사용하지 못하게 하거나 다른 사용자가 접근하지 못하게 하는 역할을 수행함
  -> init 상태라고도 의미
  -> init [모드번호] 를 입력하여 런레벨을 변경할 수 있음

예시) init S, init 5

* init() 프로세스 -> /etc/inittab 파일에 정의된 런레벨 확인 -> /etc/rc.d/rc[x].d (x 는 런레벨) 디렉터리에 나열된 스크립트를 실행하여 시스템의 운영상태 구성

* 런레벨이 의미하는 시스템 운영 상태 - 유닉스
  1) 0 : PROM(Programming Read-Only Memory) 모드
  2) S, s: 시스템 싱글 유저 모드 (싱글 유저 모드는 시스템을 유지보수하기 위한 모드로 root 사용자 암호가 필요함) , 로컬 파일 마운트 x
  3) 1 : 시스템 싱글 유저모드 , 로컬 파일 마운트 o
  4) 2 : 멀티 유저 싱글 모드 (NFS 클라이언트 모드)
  5) 3 : 멀티 유저 모드 (NFS 서버 모드), 유닉스 기본 런레벨
  6) 4 : 사용안함
7) 5 : 시스템 power off , cf : 리눅스의 경우 X 윈도우 환경으로 실행된 멀티 유저 실행 모드

8) 6 : 시스템 리부팅

-> who -r 또는 runlevel 을 사용하면 알 수 있음

* Unix 시스템의 부팅 과정
 바이오스 과정-> 부트 프로그램 과정 -> 커널 과정 -> init 프로세스 과정

1) 바이오스 과정 : 시스템의 기본적인 하드웨어의 이상 유무를 점검하고 시스템의 하드웨어 정보를 수집
2) 부트 프로그램 과정 : 하드디스크에서 커널을 읽어서 메모리에 적재, 시스템 제어권을 커널에 넘김
3) 커널 과정 : 커널이 메모리에 적재 된 후 운영체제 구동 시작, 하드웨어 점검 완료, 내부 자료구조를 초기화 하여 시스템을 운영하기 위한 부가적인 커널 모듈을 하드 디스크에서 메모리 상으로 적재함
4) init 프로세스 과정 : 커널에 의해 생성되는 첫 번째 프로세스. PID 가 1 임. init 프로세스는 모든 유닉스 프로세스의 부모 프로세스로서 부팅 과정에서 사용자가 시스템을 사용할 수 있게 해주는 초기화 작업을 담당함

* BSD 계열 -> 싱글 유저와 멀티 유저모드 두가지 지원 ,
  System V 계열 -> 런레벨로 분류 (etc/inittab 파일에 정의)

* 유닉스 시스템 종료시 주의 사항
 -> 접속중인 사용자에게 시스템의 종료를 공지하여 작업을 마무리하도록 해야함
 -> 운영 중인 서비스(프로세스) 를 안전하게 종료
 -> 하드 디스크를 갱신하여 파일 시스템의 무결성을 유지
 -> 하드디스크 동기화 필수 (sync 명령어를 사용하여 버퍼의 내용을 하드 디스크로 옮겨야 함)

* shutdown 명령은 시스템을 안전하게 종료할 때 사용하며 시스템의 런레벨을 변경할 때도 사용함

** p. 37 Sytem V 와 BSD 계열의 shudown 명령어 기재

* 사용자 계정 추가 -> useradd 명령어 사용 (root 만 사용 가능)
 - /etc/password : 사용자 계정 정보 저장
 - /etc/shadow : 암호화된 비밀번호 저장
 - /etc/group : 그룹 정보 저장

* 사용자 계정 삭제 -> userdel (root 만 사용 가능)

* 그룹 추가 -> groupadd

groupadd [-g gid] group_name

* 그룹 삭제 -> groupdel

* 파일 시스템 연결 -> mount
 - 보조기억장치에 설치된 파일 시스템을 유닉스 시스템이 인식하도록 논리적으로 연결해줌
 - /etc/fstab 파일에 정의된 모든 파일시스템을 마운트 해줌

mount [-option] [devcie | mount_point]

 mount -a
 mount /dev/cdrom /mnt/cdrom : dev/cdrom 디바이스 파일을 /mnt/cdrom 으로 마운트

* 파일 시스템 연결 해제 -> unmount

umount [-option] [devcie | mount_point]

* /etc/mtab : 시스템에 마운트된 파일시스템의 정보를 관리함.

* 하드디스크 사용량 -> du

du [-option] [directory_name]

du -a : 하위파일 정보까지 출력
du -s : 현재 디렉터리가 차지하는 총용량 확인
du -k : 사용량을 킬로바이트로 환산

* 파일시스템 용량 정보 (파일시스템의 전체 공간 및 사용 가능한 공간에 대한 정보 제공)-> df

df [-option] [file_system | file]

df /boot

* 정기적 스케줄 관리 -> cron
- cron 데몬 프로세스는 시스템에서 정기적인 작업을 지정시간에 처리하기 위해 사용함
- 일괄적으로 처리하는 작업 (야간작업), 작업에 대한 요구가 불규칙하지 않은 경우, 우선순위가 낮은 작업을 일정한 시기에 처리, 작업 빈도가 낮아서 필요 시점에 처리하여 마무리하는 것이 자원 관리상 효율적인 경우

- cron 데몬 프로세스 사용시 필요한 세가지 구성요소 -> crontab 파일(작업목록), crontab 명령(파일편집), cron 데몬 프로세스(시스템에서 기본적으로 제공해줌)

* crontab 파일 구조
- 각 행은 space 나 tab 으로 구분된 6개의 필드로 구성 됨
- 분 (0-59) / 시 (0-23)/ 일 (1-31) / 월 (1-12) / 요일 (0-6, 0:일요일) / 작업 (지정한 시간에 실행할 작업을 절대 경로로 기술하고 필요한 옵션 및 인수를 함께 나열)
- 기호구분 : ‘*’ (필드범위에 해당하는 모든 값), ‘-’ (값의 범위), ‘,’(값을 구분), ‘/’(간격값)

* crontab 파일의 제어
- 사용자 계정 별로 생성
- root 는 사용자를 지정하여 편집 가능, 일반 사용자는 자신의 crontab 만 편집 가능

crontab -e : crontab 파일을 편집
crontab -l : crontab 파일을 출력
crontabl -r : crontab 파일을 삭제

예시) 0 3 * * 0 /bin/rm -rf /tmp/* >/dev/null 2>&1

-> 매주 일요일 오전 세시에 /tmp 하위의 모든 파일을 삭제, 표준 출력은 출력되지 않도록 /dev/null 장치파일로 출력재지정, 표준에러(2) 도 표준출력(&1) 로 출력 재지정하여 표준 출력과 동일하게 /dev/null 장치파일로 출력하여 버려짐

*/dev/null 장치 파일 : 특수한 장치파일로 해당 파일로 출력하면 이를 모두 버리는 역할을 함. 윈도우의 휴지통과 유사한 역할

* crontab 명령 접근 제어
- /etc/cron.allow : 화이트리스트 방식 (등록되어있는 사용자만 편집 가능)
- /etc/cron.deny : 블랙리스트 방식 (등록된 사용자를 제외하고 모든 사용자가 crontab 명령 실행 가능)
-> 두 파일다 없을 경우 root 만 crontab 명령 사용가능

* 일시적 스케줄 관리 -> at
- 정해진 시간에 한번만 실행함
- 한번 처리된 작업은 작업 목록에서 삭제 됨

* 패스워드 저장 정책
- /etc/passwd 파일에 저장 /etc/shadow 파일에 저장
- Passwd 파일의 두 번째 필드에 x 로 저장 후 shadow 파일에서 비밀번호 따로 관리

* /etc/passwd 파일 구조

user_account:user_password:user_ID:group_ID:comment:home_directory:login_shell

* /etc/passwd 파일에서 유의해야하는 보안 사항

- root권한에 대해 원격 접속 금지. Root권한이 필요하다면 일반계정에서 su 명령어를 사용하여 스위칭해서 작업
- UID/GiD 가 0일 경우 root 권한을 사용할 수 있으므로 주의
- 로그인이 불필요한 계정에 대해서는 로그인을 금지시키도록 해야함
  -> 로그인을 금지시킬 계정의 login_shell을 /sbin/nologin 또는 /bin/false 로 설정

* /etc/shadow
- 일반적으로 root만 읽을 수 있도록 설정되어 있음
- Aging 정보(시간의 흐름에 따른 패스워드 관리정책)와 암호화된 패스워드. 정보가 기록

* /etc/shadow 파일 구조

user_account:encrypted_password:last_change:minlife:maxlife:warn:inactive:expires

- last_change : 마지막으로 패스워드를 변경한 날 (1970년 1월 1일 기준)

- minlife : 패스워드 최소 사용기간 (최소 사용기간이 없으면 최근 암호 기억을 무력화하여 익숙한 패스워드를 재사용할 문제점이 있음)
- Maxlife : 패스워드 최대 사용기간 (변경한날 이후부터 패스워드 만료 일수를 의미하며 최대 사용기간이 없으면 패스워드 유출발생 시 공격자의 계속된 접근을 야기할 수 있으므로 90일로 설정할 것을 권장함).

- warn : 패스워드 만료 이전 경고 일수를 의미
- Inactive : 패스워드가 만료된 이후 계정이 잠기기전까지 비활성 일수. 해당 비활성 기간에 패스워드를 변경하지 않으면 계정이 잠김 (유닉스의 경우에는 패스워드 변경 여부가아니라 로그인 여부로 확인)
- Expires : 계정 만료일 설정으로 1970년 1월 1일을 기준으로 일수로 표기

* /etc/shadow 의 encrypted_password 필드 구성

IDSalt$enrypted_password

- ID : 암호화에 적용된 일방향 해시 알고리즘을 식별하기 위함.
1 -> MD5, 2 -> BlowFish, 3-> SHA-256, 4->SHA-512

- salt : 패스워드 암호화 강도를 높이기 위한 임의의 랜덤 값, 레인보우 테이블 공격에 효과적으로 대응 가능
- encrpyted_password : * 면 패스워드가 잠긴 상태로 로그인 불가 (별도의 인증 방식을 통해서는 가능), !! 면 계정 생성 후 비밀번호를 갱신하지 않은 상태로 모든 로그인이 불가, 빈값이면 패스워드가 설정되지 않은 상태로 비밀번호 없이 로그인 가능

* passwd -l 계정명 : 패스워드 잠금 / passwd -u 계정명 : 패스워드 잠금해제

* 유닉스 /etc/shadow 파일
 - NP : No Password . 로그인할 수 없는 계정. 시스템 및 애플리케이션 계정
 - *LK* : Lock 의 의미로 패스워드가 잠긴 상태로 로그인할 수 없음
 - 빈값 : 패스워드가 설정되지 않은 상태로 패스워드 설정과정이 진행 됨

* 패스워드 저장 정책 변경
 - pwconv : shadow 에 비밀번호 저장하도록 변경
 - pwunconv : passwd 파일 내 계정정보와 함께 비밀번호를 저장하도록 변경

* 프로세스 실행 권한 -> 프로세스의 자원에 대한 접근 권한 판단용
 - RUID : 프로세스를 실행시킨 사용자의 UID
 - RGID : 프로세스를 실행시킨 사용자의 GID
 - EUID : 프로세스가 실행 중인 동안에만 부여되는 UID (접근권한 판단용)
 - EGID : 프로세스가 실행 중인 동안에만 부여되는 GID (접근권한 판단용)

* SUID(Set-UID), SGID(Set-GID)는 프로세스가 실행 중인 동안에 일시적으로 자원에 ‘ 실행파일의 소유자 권한’ 으로 접근할 수 있도록 하는 설정, 별도로 설정하지 않고 프로세스를 실행시키면 RUID / EUID , RGID / EGID 가 동일하게 설정 됨 -> 간단하게 얘기하면 실행시킨 사용자의 UID/GID 로 적용 됨
* SUID / SGID 가 설정된 프로세스는 RUID, RGID 는 실행시키는 사용자를 기준으로 적용 되고 , EUID,EGUID 는 실행파일 소유자를 기준으로 적용 됨

-> S-UID / S-GID 설정 ok : 프로세스 실행자 (R) / 프로세스 소유자 (E)
-> S-UID / S-GID 설정 X : 프로세스 실행자 (R, E)

* S-UID : s / 4
* S-GID : s / 2
* Sticky-bit : t / 1

=> root소유의 자원에 대해 일반 사용자의 직접 접근을 차단하면서 기능상 필요한 부분만을 접근하도록 허용할 필요가 있는 경우, 별도의 root 소유 프로그램을 통해 suid/sgid 를 설정하여 사용하는 것이 보안적인 측면과 운영적인 측면에서 매우 효율적임
=> root 권한이 필요 없는 프로그램에 소유주가 root로 되어 있으면서 setuid 가 설정되어 있으면 보안상 취약하므로 주기적으로. Setuid, setgid 가 설정된 프로그램을 확인할 필요가 있음
(find / -user root -type f \(-perm -4000 -o -perm -2000 \) -exec ls -al {} \;)
=> 소유권이 root 인데 setuid, setgid 가 설정되어 있는 대표적인 파일로 passwd 명령어가 있음

- Chmod -s 실행파일명을 통해 setuid, setgid 권한을 제거할 수 있음

*. 디렉터리 접근권한 (sticky-bit)
공유 파일에 대한 보안을 목적으로 시스템에 있는 모든 사용자가 자유롭게 파일과 디렉터리를 생성할 수 있지만 파일 삭제 및 파일명 변경은 소유자 또는 root 만 가능하게 제한함

- /var/tmp , /tmp
- 리눅스의 경우 chmod o+t /tmp, 유닉스의 경우 chmod u+t /tmp 로 설정함

* 보안 쉘 (ssh): 암호 통신을 이용하여 네트워크 상의 다른 컴퓨터에 접속하여 원격으로 명령을 실행하거나 파일을 조작하는 응용 프로그램 또는 프로토콜을 의미한다.
-> 암호화된 원격 터미널 서비스 / 파일 송수신 서비스를 제공함
-> 22/tcp 사용함
-> rsh, login, telnet, ftp 의 대안

* 슈퍼 데몬 : 효율적인 서버 자원의 활용을 위해 공통적인 부분을 처리하는 슈퍼데몬을 만들어 개별 서비스를 등록하게 하여 클라이언트 요청은 슈퍼 데몬이 처리하고 적절한 개별 서비스를 호출해 주는 방식
- stand-alone : 개별 서비스 별로 서버 프로세스(데몬) 이 동작함. 속도는 빠르지만 서버 자원을 많이씀
- inetd(xinted)방식 : 슈퍼 데몬을 이용하여 개별 서비스를 동작시키는 방식으로 상대적으로 속도가 느리지만 서버자원을 절약할 수 있음 . 최초 실행시 /etc/inetd.conf 의 파일 정보를 참조해서 서비스할 프로그램들에 대한 정보를 얻음 . 서비스할 프로그램들의 특징을 이 파일에 정의하면 됨
- tcpd(TCP Wrapper) 서비스와 연동하여 서비스별 호스트 접근 제어도 가능
- 리눅스는 xinetd 데몬을 주로 사용 . Inetd 에서 보안과 자원 관리가 향상된 버전임)

* inetd.conf 파일의 구조

서비스명 소켓타입 프로토콜 플래그 사용할사용자계정 실행경로명 실행인수

예시)
ftp stream tcp nowait root /usr/sbin/in.ftpd in.ftpd -l a

->서비스명은 /etc/services 파일에 등록된 포트번호를 참조하여 서비스할 프로세스의 포트를 결정함
-> 소켓 타입은 tcp 면 stream, dup 면 datagram
-> 프로토콜은 /etc/protocols 파일에 주어진 프로토콜 중 사용 가능한 프로토콜을 설정함. /etc/services 파일에 설정한 프로토콜과 일치해야함
-> 플래그는 nowait(서비스요청시 즉시 처리). / wait(요청 처리 완료 후 다음 요청. 즉 기다렸다가 처리) 로 구분

-> 실행경로명은 절대경로로 작성
-> 실행인수의 첫번째는 응용 프로그램 자신의 이름이 됨

* /etc/protocols : 인터넷상에서 사용하는 프로토콜과 프로토콜 식별 전호를 정의해둠
* /etc/services : 인터넷상에서 사용하는 다양한 서비스에대한 포트/프로토콜정보를 정의함. well-known(0-1023), registerd port(1024-49151), dynamic port(49152-65535)

=> 둘 다 IANA 에서 관리함

* simple tcp (dos 공격에 취약) : echo(7/tcp), discard(9/tcp), daytime(13/tcp), chargen(19/tcp)
* R 계열 서비스 : rlogin, rsh, rexec 등 r 계열 서비스는 인증없이 관리자의 원격 접속을 가능하게 하는 명령어들로 이기종 운영체제 간 백업 등의 용도로 사용되는 경우가 있으나 보안상 매우 취약하므로 사용하지 말아야함

* 불필요한 rpc(remote procedure all) 서비스 : rpc, cmsd, resuresd 등 분산 환경에서 서버 응용 프로그램에 접근하여 작업 호출을 할 수 있는 서비스로 버퍼 오버 플로우등 다수의 취약점이 존재하여 침해사고 발생 위험이 있으므로 서비스를 중지해야함
* Finger, tftp, talk 도 중지

=> inted.conf 파일에서 취약한 서비스를 grep 하여 # 처리 후 inted 서비스 재시작 필요

* TCP Wrapper

- 외부에서 들어오는 클라이언트에 대해 접근 통제 기능 제공

- /etc/hosts.allow 와 /etc/hosts.deny 파일에 정의된 호스트정보를 기주느로 함
- 접근제어시 hosts.allow -> hosts.deny 순으로 파일을 확인함. Default 는 모든 접근 허용
- 사용전에는 inetd.conf 에 실행경로가 /usr/bin/in.telnetd 이지만 사용 후에는 /usr/sbin/tcpd 로 바뀜

* hosts.allow / hosts.deny 파일 구조

서비스리스트 : 클라이언트 리스트 [: 쉘 커맨드]

-> 일반적으로 hosts.deny 파일은 ALL:ALL 로 설정되어 있음 (모든 호스트로부터 모든 서비스 차단. 이렇게하면 hosts.allow 에 있는 걸 기준으로 접근허용을 할 수 있음)

p.55 에 hosts.allow / hosts.deny 예제 있음

-> 쉘 커맨드는 twist 와 spawn 으로 나뉨. 일반적으로 hosts.deny 파일에 설정하며 차단된 호스트에게 즉시 통보나 경고 메세지를 보낼 수 있음. Twist 를 사용하면 클라이언트에게 전송하고 spawn 은 클라이언트에게 전송하지 않음

* xinetd 슈퍼데몬 : inetd의 비효율적인 자원 관리와 보안성 문제를 개선한 슈퍼 데몬으로 TCP wrapper 기능 뿐만아니라 자체적인 서비스별 접근제어기능을 제공함
* /etc/xinetd.conf : 모든 서비스에 적용되는 xinetd 설정 파일
* /etc/xinetd.d/서비스별 설정 파일 : 개별 서비스에 적용되는 xinetd 파일

* xinted 설정파일 형식 (일반 설정)
- service : 서비스 이름 설정
- disable : 서비스 실행 여부 결정 (no : 실행, yes : 실행 안함)
- socket_type : 서비스 소켓 유형 설정 (stream : tcp 기본 설정 / dgram : dup 기본 설정).

- wait : 서비스 요청 처리 중 다음 요청이 발생하면 이를 대기시키거나 즉사 처리할지 결정 (yes : 대기 / no : 즉시처리 -> tcp default)
- user : 서비스 실행 사용자
- server : 서비스 실행 파일 경로 설정

* 설정 파일 형식 (접근 제어 관련 설정).
 - only_from : 접근을 허용할 특정 IP주소 또는 IP주소 대역 (대역은 CIDR 표기 사용)
 - no_access : 접근을 차단할 특정 IP주소 또는 IP주소 대역
 - access_times : 접근을 허용할 시간 (24시간 기준 . 예를 들면 09:00-12:00 13:00-18:00)
 - cps : 초당 연결개수를 제한하기위한 초당 최대 연결개수를 설정 (두개의 인수로 구성됨. cps = 50 10 라고하면 초당 최대 연결개수를 50개로 제한하고 초과 연결 요청시 10초간 연결을 제한한 후 다시 서비스를 개시하라는 의미)
 - instances : 동시에 서비스할 수 있는 서버개수를 제한하기 위한 최대 서버 개수 설정 (서비스 프로세스 설정)
 - per_source : 출발지 IP 를 기준으로 서비스 연결개수를 제한하기 위한 출발지 IP별 최대 연결개수를 설정 (특정 출발지 IP 의 클라이언트로부터 과도한 서비스 연결 요청이 발생하는 상황을 통제하기 위함)

* 설정 파일 형식 (로그 관련 설정)
- log_on_failure : 서버 접속 실패시 로그 파일에 기록할 내용을 설정함
- log_on_success : 서버 접속 성공 시 로그파일에 기록할 내용을 설정함

* PAM(장착형 인증 모듈) - Pluggable Authentication Modules

- 리눅스 시스템 내에서 사용되는 각종 애플리케이션 인증을 위해 제공되는 다양한 인증용 라이브러리를 의미하며 /lib/security 또는 /usr/lib/security 하위에 있음 . /etc/security 에는 PAM 모듈 실행에 필요한 추가 설정파일이 있음. /etc/pam.d 에 PAM 을 사용하는 응용프로그램별 설정파일이 있음 (파일명은 응용프로그램의 서비스명과 동일)
- 장점 : 인증방식 및 정책의 유연성과 중앙 통제가 가능, 기존 응용프로그램을 수정하지 않고도 새로운 인증 서비스 모듈을 추가하여 사용가능

* PAM 설정파일 형식 (예시) /etc/pam.d/remote)

Type Control module-pam module-arguments

-> type 은 pam 모듈의 종류를 의미함 (account / auth / password / session) 으로 구분 됨

-> control 은 각 모듈 실행 후 성공 또는 실패에 따른 pam 라이브러리의 행동을 결정함 (requisite -> 인증 실패시 즉시 인증 거부 / required -> 인증에 실패 하더라도 다음 라인의 모듈을 실행하지만 최종 결과는 인증 실패 / sufficient -> 이전에 있는 요청들이 실패하더라도 여기서 성공하면 pam 인증 성공)

-> module-Path 는 pam에서 사용할 실제 모듈 파일이 위치한 경로ㅡㄹ 의미함 (lib/security에 *.so 형식으로 존재)
-> module-arguments 는 모듈에 전달되는 인수를 의미함

* pam 활용 예시 - 시스템 취약점 분석/평가 항목

- root 계정의 원격 접속 제한 -> /etc/securetty 파일 (pam_securetty.so 모듈이 사용하는 파일로 너미널 접속시 root 접근 제한을 설정함)에 등록되어 있는 터미널이 아니면 root 접속이 불가하도록 pam 설정
  -> /etc/sercuretty 파일의 pts 터미널을 모두제거

Cf. tty(terminal-teletype) 사용자가 서버콘솔에 직접 로그인 (서버에 직접 붙어서 모니터랑 키보드를 연결해서 접속)/
pts(pseudo-terminal) telnet이나 터미널을 통해 접속하는 가상 터미널

* pam 활용 예시 - 유닉스/리눅스별 root 계정 원격 접속 제한 설정

- SOLARIS
#cat /etc/default/login
CONSOLE=/dev/console -> 여기에 #을 붙이면 외부에서 root 접속 가능, 안붙이면 콘솔에서만 접속 가능

- LINUX
#cat /etc/pam.d/login
auth required /lib/security/pam_security.so

#cat /etc/securetty -> pts/0 ~ pts/x 관련설정이 존재하지 않음

-AIX
#cat /etc/security/user
rlogin= false -> 외부접속 불가. true로 사용하면 외부접속 가능

-HP-UX
#cat /etc/securetty
console -> 주석처리하면 외부에서 root 계정으로 접속가능, 안하면 외부 접속 불가 (콘솔 접속만 가능함)

* pam 활용 예시 - ssh

telnet, ftp 를 사용하여 원격 접속 또는 파일 송수신을 할 경우 평문 통신을 하기 때문에 스니핑에 취약함. ssh를 사용하면 암호통신을 이용하여 네트워크상의 다른 컴퓨터에 접속하여 원격으로 명령을 실행할 수 있음.

ssh 를 통해 원격 터미널 서비스와 파일 송수신을 할 수있음(telnet / ftp 대체)

- 설정방법
ls -al /etc/ssh/sshd_config 파일에서
PermitRootLogin 항목을 no로 설정

* PAM 활용 예시 - 시스템 취약점 분석/평가 일부 항목
- 무작위 대입공격이나 사전대입공격발생시 암호입력 실패 횟수를 적절하게 제한하고 공격시간을 지체시켜 패스워드 유출 위험을 줄일 수 있음
-> 계정 잠금 임계값 지정(권장 5회이하). 실패하면 패스워드 잠금
-> pam_tally2.so 모듈 이용
-> /etc/pam.d/system-auth 서비스 설정 파일에
pam_tally2 모듈 추가 (deny, unlock_time)
-> pam_tally2 -u 계정명 명령을 통해 로그인 실패 횟수 확인
-> pam_tally2 -u 계정명 -r 명령을 통해 해당 계정의 실패 횟수 초기화

* PAM 활용 예시 - 유닉스/리눅스 시스템별 계정 잠금 임계값 설정

-SOLARIS
#cat /etc/default/login
RETRIES=5

#cat /etc/security/policy.conf -> 5.9 버전 이상일 경우 확인
LOCK_AFTER_RETIRES=YES

-LINUX
#cat /etc/pam.d/system-auth
auth required /lib/security/pam_tally.so deny=5 unlock_time=120 no_magic_root

account required /lib/security/pam_tally.so
no_magic_root
reset

-> no_magic_root를 사용하면 root계정에 대해서는 패스워드 잠금 설정을 적용하지 않음
-> reset 은 접속시도 성공시 실패한 횟수를 초기화 함

-AIX
#cat /etc/security/user
loginretries=5

-HP-UX
#cat /tcb/files/auth/system/default
u_maxtries#5

#cat /etc/default/secuirty -> 11.v3 이상이면 확인
AUTH_MAXTRIES=5

* PAM 활용 예시 - 시스템 취약점 분석 /평가 일부항목 -> su 관련 취약점

- su 명령어 사용이 허용된 사용자만 root 계정으로 접속 할 수 있도록 사용함
- pam_wheel.so 모듈 이용
-> 직접 /etc/group 파일을 수정하거나 usermod -G wheel 계정명을 입력하여 그룹 변경
-> /etc/pam.d 디렉터리에 있는 su 설정 파일에 설정함

* PAM 활용 예시 - 유닉스/리눅스 시스템별 su 제한설정
- wheel 그룹 (su 명령어 사용 그룹) 및 그룹 내 구성원 존재 여부 확인
#cat /etc/group
- wheel 그룹이 su 명령어를 사용할 수 있는지 설정 여부를 확인함
-> SOLARIS : ls -a /usr/bin/su
chgrp security su
chmod 4750 su

-> AIX : cat /etc/security/user -> default의 “sugroup=staff” 설정 확인
-> HP-UX : vi /etc/default/security -> SU_ROOT_GROUP=wheel 설정 확인

-파일 권한을 확인함 : ls -l /usr/bin/su (파일권한이 4750 이면 양호하다고 판단

cf. LINUX PAM 모듈 이용시 -> wheel 그룹 및 그룹내 구성원 정보를 확인하고 허용 그룹의 설정 여부를 확인함

cat /etc/group
cat /etc/pam.d/su -> 결과가 auth required /lib/security/pam_wheel.so debug group=wheel 또는 auth required /lib/security/$ISA/pam_wheel.so use_id 여야함

* PAM 활용 예시 - sudo 명령을 이용한 root 권한 부여
- sudo 는 다른 사용자 계정 권한으로 명령어를 실행하고자 할 때 사용하는 명령임
- su 는 root 의 비밀번호를 알아야함
- sudoers(/etc/sudoers) 에서 sudo명령에 대한 설정을 할 수 있음

sudo -u 실행 권한계정명 명령어 -> -u 실행권한계정명을 생략하면 root 권한으로 실행 됨

- sudoers 파일
계정명 호스트명=(실행권한계정명) [NOPASSWD:]명령어

-> NOPASSWD를 통해 계정의 비밀번호를 물어보지 않도록 설정할 수 있음
->실행권한계정명을 생략하면 root권한이 부여 됨. 여기에 설정한 계정의 권한으로 실행 됨

예시)

algisa ALL=(ALL) ALL -> algisa 는 sudo 명령어를 통해 모든 호스트에서 root 권한(모든권한)으로 모든 명령어를 실행할 수 있음

algisa ALL=(root,kiwi99,kiwi88) ALL -> algisa는 sudo명령을 통해 모든 호스트에서 root, kiwi99, kiwi88 계정 권한으로 모든 명령을 실행할 수 있음

*시스템 로그 설정과 관리
- 유닉스 : /var/adm , 리눅스 : /var/log -> 리눅스에서 설정 파일은 /etc/syslog.conf 에 해당함 (시스템 로그 파일들의 위치를 지정)

* utmp(x) - linux, unix, sunOS
- 현재 로그인한 사용자의 상태정보
- binary -> w, who, finger 명령어로 확인 가능
- LInux -> /var/run/utmp , Unix -> /var/adm/utmpx

p.71 에서 명령어별 출력 결과 확인 가능

* wtmp(x) - linux, unix, sunOS
- 사용자의 성공한 로그인/로그아웃 정보
- 시스템의 Boot/shutdown 정보 -> reboot 사용자를 통해 확인
- binary -> last, last 계정명 (계정명, 터미널타입, 접속주소, 로그인시간, 로그아웃시간 확인)
- linux -> /var/log/wtmp, unix -> /var/adm/wtmpx

* lastlog 파일 - linux, unix, sunOS
- 가장 최근에 성공한 로그인 기록을 담고 있음
- binary -> lastlog(linux), finger(linux, unix/sunOS)
- linux ->/var/log/lastlog, unix -> /var/adm/lastlog

- lastlog -u 계정명을 통해 사용자를 지정해서 최근 접속 기록 확인 가능, lastlog -t 일수를 통해 해당 일수 이내에 접속한 기록을 확인할 수 있음

  finger 계정명 으로 사용하면 지정한 계정의 마지막 로그인 정보 확인 가능

* btmp(linux), loginlog(unix(sunOS)) 로그 파일
- 실패한 로그인 시도에 대한 기록을 담고 있음
- Linux -> /var/log/btmp, binary 파일 -> lastb (실패한 모든 로그), lastb 계정명을 통해 특정 계정을 지정할 수 있음
- Unix(SunOS) -> /var/adm/loginlog -> 텍스트파일 -> vi 로 확인 (5회 이상 실패시에 대해서만 로그가 남겨짐)

* sulog 파일
- su 명령어를 사용한 결과를 저장한 파일로 SunOS 를 포함한 Unix 계열에서만 볼 수 있음
- /var/adm/sulog -> 텍스트 파일 -> vi 편집기 -> 열어봤을 때 + 는 su 명령어의 성공, - 는 실패를 나타냄

cf. 리눅스는 /var/log/secure 로그 파일에 su 명령의 결과가 남음

* acct/pacct 로그 파일
- acct/pacct 로그 파일은 시스템에 로그인한 모든 사용자가 로그아웃 할 때 까지 입력한 명령어와 터미널의 종류, 프로세스 시작 시간등을 저장한 로그임
- linux -> /var/account/pacct -> binary -> accton /var/account/pacct 명령어 실행 필요 (기본으로 생성되지 않으므로 명령어 실행이 반드시 필요함)
- unix -> /var/adm/pacct -> binary -> /usr/lib/acct/accton /var/adm/pacct 명령 실행 필요 (기본으로 생성 되지 않으므로 명령어 실행이 반드시 필요)

- 내용 확인은 lastcomm 으로 확인 -> 실행한 명령어, 플래그(S -> 슈퍼유저로 root 에 의해 실행 됨 ,F -> exec 없이 fork 에 의해 실행함 ,X-> SIGTERM 시그널에 의해 종료 됨), 실행한 사용자명, 터미널 타입, 프로세스 시작 시간 정보

* history 로그 파일
- 계정별로 실행한 명령어에 대한 기록을 저장한 파일로 계정별 홈 디렉토리에 존재한다
- 로그파일은 '.쉘종류_history' 형식으로 생성
- 텍스트 파일 -> vi , 또는 history 로 확인 가능
- 예시) cat .bash_history | head -5

* secure 로그 파일 (linux)
- 사용자/그룹생성/삭제, 로그인 등의 사용자 인증에 대한 정보를 기록. 서버 보안에 아주 민감
- su 내역, 원격 접속 내역 저장
- linux -> /var/log/secure

* message 로그 파일 (linux)
- 리눅스의 기본적인 시스템 로그 파일
- 데몬 실행상황, 내역, 사용자 접속정보, Tcp wrapper 접근 제어
- /var/log/messages

* dmesg 로그 파일 (linux)
- 리눅스가 부팅될 때 출력되는 모든 메시지를 기록함
- 텍스트 파일 -> 텍스트편집기, dmesg
- /var/log/dmesg

* boot.log 파일 (linux)
- 리눅스가 부팅될 때 파일시스템에 대한 체크, 서비스 데몬들의 실행 상태등을 기록하고 있는 로그 파일, 성공/실패 여부를 확인 가능
- /var/log/boot.log

* xferlog (linux)
- ftp 서비스 로그파일 (vsFTP, proFTP, ProFTP 포함)
- FTP 접속일시, 원격 호스트 사용자 정보, 업로드/다운로드 파일정보 확인 가능
- /var/log/xferlog

p 79 에 상세 실습내용 참고

-> 전송 유형 : b (바이너리) / a (아스키/텍스트)
-> 액션 플래그 : - (액션없음) / C (파일이 압축됨) / U (압축된 파일이 해제됨) / T(파일이 아카이브됨 -tar 로묶임)
-> 전송방향 : i (서버로부터 파일 업로드) / o(서버로부터 파일다운로드) / d (서버에 있는 파일 삭제)
-> 접근 방식 : r (로컬 시스템 계정으로 접근) / a (익명 계정으로 접근) / g (게스트 계정으로 접근)
-> 인증방법 : 0 : 없음 / 1 : RFC 931 인증
-> 인증된 사용자 ID : * 인 경우 이용이 불가능한 상태
-> 완료 상태 : c : 전송성공 / i : 전송실패

* cron 로그 파일 (linux)
- 시스템에서 동작중인 cron 데몬이 수행한 정기적인 작업을 기록한 로그파일
- /var/log/cron

* maillog 로그 파일
- 시스템에서 동작중인 메일서버의 메일 송수신 작업을 기록한 로그 파일
- /var/log/maillog
* syslog 설정 및 관리
- 유닉스 시스템은 syslog 표준 인터페이스(API) 를 통해 커널 및 응용 프로그램에 의해 발생하는 로그를 체계적으로 생성하고 관리함
- 동작 방식은 커널 및 응용 프로그램이 syslog API 를 통해 로그를 생성하면 syslogd 데몬 프로세스가 syslog.conf 설정 파일을 참조하여 지정한 로그 파일, 콘솔(Console) 또는 외부 서버(Remote Server)등에 로그를 기록함

-> syslog API 가 로그 생성 -> syslogd 데몬프로세스가 로그 기록

- /etc/syslog.conf : syslogd 가 실행될 때 참조되는 로그파일
- syslog.conf 형식

facillity.priority; facillity.priority …
action(logfile-location)

-> facility : 로그 생성서비스 (*, auth/ahtipriv, cron, daemon, kern, lpr, mail, news, uucp, user, syslog, local0~local7)
-> priority : 로그 수준 (emerg , alert, crit, err, warning, notice, info, debug)
-> action : 로그를 어디에 남길 것인지 결정. 로그파일, 콘솔, 원격 로그 서버, 특장사용자등에 로그를 남김 (로그파일, console/dev -> 콘솔, 원격로그서버 -> @호스트주소, user -> 지정된 사용자의 스크린, * -> 현재 접속해 있는 모든 사용자의 스크린)

- 최근 리눅스에서는 syslog 를 개선한 rsyslog 를 주로 사용
-> rsyslogd + /etc/rsyslog.conf 사용

- syslog 는 기밀성, 무결성, 가용성 등 정보보호 특성을 고려하지 않고 개발됨
-> RFC 3195 에서 TCP, BEEP 사용 권장

* BEEP : the Blocks Extensible Exchange Protocol 의 약자로 연결지향적이고 비동기적인 연결을 위한 응용프로그램 프로토콜 프레임 워크로 내부적으로 인증, 프라이버시, 재전송을 통한 신뢰성 등을 보장함

* 로그 모니터링 - tail -f /var/log/messages (단, 바이너리 형식은 사용불가)

* 로그 파일 순환
- /usr/sbin/logrotate -> logrotate 데몬 위치 및 데몬 프로그램
- /etc/logrotate.conf -> logrotate 데몬 설정 파일

- /etc/logrotate.d -> logrotate 를 적용할 프로세스의 개별 설정 파일이 위치한 디렉터리
- /var/lib/logrotate.status -> logrotate 를 수행한 작업 내역을 보관한 파일
- /etc/cron.daily/logrotate -> logrotate는 주기적으로 실행되어야 하므로 cron 데몬에 의해 일 단위로 실행 된다.

* logrotate.conf
- daily/monthly/weekly : 일단위, 월단위, 주단위 로그파일 순환
- rotate n : 순환 로그파일의 개수를 n 개 설정
- create [퍼미션] [소유자] [소유그룹] : 순환시 새롭게 로그파일을 생성하며 퍼미션, 소유자, 소유그룹을 지정할 수 있음 (예시 - create 600 root root)
- dateext : 로그파일의 확장자로 날짜를 붙여서 보관
- compress, nocompress : 압축여부 결정
- size n : 지정한 크기가 되면 로그 파일 순환 (예시 - 100K, 100M)
- include /etc/logroate.d : 해당 디렉터리에 있는 데몬/프로세스 설정 파일을 포함함

p.86 에 httpd 로그 설정파일 예시 있음

* 시스템 해킹 - 버퍼 오버 플로우 공격
- 버퍼 오버플로우는 연속된 메모리 공간을 사용하는 프로그램에서 할당된 메모리의 범위를 넘어선 위치에 자료를 읽거나 쓰려고 할 때 발생
- 버퍼 오버플로우가 발생하면 프로그램의 오작동을 유발시키거나 악의적인 코드를 실행시켜 공격자 프로그램을 통제할 수 있는 권한을 획득할 수 있음

* 스택 버퍼오버 플로우
- 스택 : 함수 처리를 위해 지역변수 및 매개변수가 위치하는 메모리 영역

- 정의된 버퍼의 한계치를 넘는 경우 버퍼 오버플로우가 발생하여 복귀 주소를 변경하고 공격자가 원하는 임의 코드를 실행함

* 힙 버퍼 오버플로우
- 힙 : 사용자가 동적으로 할당하는 메모리 영역
- 최초정의된 힙의 메모리 사이즈를 초과하여 문자열등이 저장되는 경우 발생, 데이터와 함수 주소등을 변경하여 공격자가 원하는 임의 코드를 실행함

* strcpy 대신 strncpy를 사용하자. strncpy 는 문자열의 길이를 제한해서 버퍼오버플로우에서 안전하다

* 스택버퍼오버플로우 대응기술
- 스택가드 (카나리 단어 기법, 가드 ->카나리로 암기) : 메모리상에서 프로그램의 복귀주소와 변수사이에 특정 값을 저장해 두었다가 그 값이 변경되었을 경우 오버플로우로 판단하여 프로그램 실행을 중단 시킴
- 스택쉴드 : 함수 시작시 복귀주소를 global RET 라는 특수 스택에 저장해 두었다가 함수 종료시 저장된 값과 스택의 RET값을 비교하여 다를경우 오버플로우로 판단하여 프로그램을 중단 시킴
- ASLR(Address Space Layout Randomization) : 메모리 공격을방어하기 위해서 주소 공간 배치를 난수화 하는 기법. 실행시마다 메모리주소를 변경시켜 악성코드에 의한 특정주소 호출을 방지함

* 레이스 컨디션 공격
- 레이스 컨디션은 둘 이상의 프로세스나 스레드가 공유자원에 동시에 접근할 때 접근하는 순서에 따라 비정상적인 결과가 발생하는 조건/상황을 말한다
- 실행되는 프로세스가 임시 파일을 만드는 경우 악의적인 프로그램을 통해 그 프로세스의 실행 중간에 끼어들어 임시파일을 목적파일(심볼릭파일)로 연결하여 악의적인 행위를 하는 것
- setuid가 설정되어 있다면 중요자원에 root 로 접근하는 심각한 문제가 발생함

* 레이스 컨디션 대응 방안
- 가능하면 임시파일을 생성하지 않음
- 파일 생성시 이미 동일한 파일이 존재하는 경우 파일 생성 또는 쓰기를 금지함
- 사용하고자 하는 파일에 링크가 걸려 있으면 실행을 중단함
- Umask 를 022 정도로 유지

* 포맷 스트링 공격
- 포맷스트링은 C 언어의 printf() 등의 함수에서 사용되는 문자열의 입/출력 형태를 정의 하는 문자열로 서식 문자열이라고 표현함
- 포맷 스트링을 인자로하는 함수의 취약점을 이용한 공격으로 외부로부터 입력된 값을 검증하지 않고 입출력 함수의 포맷 스트링을 그대로 사용하는 경우 발생할 수 있는 취약점
- 공격자는 포맷스트링을 이용하여 취약한 프로세스를 공격하거나 메모리 내용을 읽거나 쓸 수 있다. -> 취약한 프로세스의권한을 획득하여 임의의 코드를 실행할 수 있다

* 포맷스트링 공격 대응방안
- 포맷 스트링을 함수의 입력 파라미터로 사용하지 않음

* root 이외의 UID 가 0 이 되지 않도록 금지
- root 이외의 계정이 uid=0인 경우 0이 아닌 적절한 uid를 부여하거나 불필요한 경우 해당 계정을 삭제함
- AIX : chuser id=2016 algisa (100 이상 설정)
- SOLARIS, Linux, HP-UX : usermod -u 2016 algisa (linux 는 500 이상, 나머지는 100 이상)

* 패스워드 복잡성 설정 필요 - 영문대문자/소문자, 숫자, 특수무자를 조합하여 2종류 조합시 최소 10자리 이상, 3종류 이상을

조합시 최소8자리 이상으로 구성, 패스워드 자주바꾸고 시스템마다 다른 패스워드 사용 권장

* 패스워드 최소길이 설정 - 패스워드 무차별 공격이나 패스워드 추측공격에 대응하기 위해 최소길이 설정 필요

- SOLARIS
#cat /etc/default/passwd
PASSLENGTH=8

- linux
#cat /etc/login.defs
PASS_MIN_LEN 8

-AIX
#cat /etc/security/user
minlen=8

-HP-UX
#cat /etc/default/security
MIN_PASSWORD_LENGTH=8

-> 일반적으로는 패스워드 최소길이를 8자 이상 설정하지만 국가기관은 9자 이상의 길이로 설정해야함

* 패스워드 최대 사용기간 설정 - 패스워드 최대 사용기간을 설정하지 않으면 기간 경과 후에도 유출된 패스워드로 접속할 수 있음, 주기적으로 변경이 필요함

- SOLARIS
#cat /etc/default/passwd
MAXWEEKS=12 (주단위)

- linux
#cat /etc/login.defs
PASS_MAX_DAYS 90(일단위)

- AIX
#cat /etc/security/user
maxage=12 (주단위)

- HP-UX
#cat /etc/default/security
PASSWORD_MAXDAYS=90(일단위)

-> 90일로 설정함 (12주)

* 패스워드 최소 사용기간 - 패스워드 사용기간을 설정하지 안ㅇ흐면 사용자에게 익숙한 패스워드로 변경할 수 있으며 재사용함으로써 패스워드의 정기적인 변경이 무의미해짐

- SOLARIS
#cat /etc/default/passwd
MINWEEKS=12 (주단위)

- linux
#cat /etc/login.defs
PASS_MIN_DAYS 90(일단위)

- AIX
#cat /etc/security/user
minage=12 (주단위)

- HP-UX
#cat /etc/default/security
PASSWORD_MINDAYS=90(일단위)

* 패스워드 파일 보호 - 패스워드 정보를 평문으로 저장하는 경우 정보 유출피해가 발생할수 있으므로 패스워드를 암호화하여 보호 해야함

-> /etc/shadow 파일에 저장 (권한이 있는 사용자들만 읽을 수 있도록 저장)

- SOLARIS, LINUX

-> /etc/shadow 파일 존재 확인, /etc/passwd 파일의 두번째 필드가 x인지 확인

#cat /etc/passwd

- AIX : 기본적으로 /etc/security/passwd 파일에 패스워드를 암호화하여 저장함

- HP-UX

-> /tcb 디렉터리 존재 확인

-> /etc/passwd 파일 내 두 번째 필드가 x 표시 되는지 확인

-> trustmode 로 전환할경우 패스워드를 암호화하여 /tcb/files/auth 디렉터리에 계정 이니셜과 계정 이름에 따라 파일로 저장

* 세션 타임아웃 설정 - 계정이 접속된 상태로 방지하면 권한이 없는 사용자에게 중요 시스템이 노출되어 악의적인 목적으로 사용될 수 있으므로 일정 시간이 후 어떤 이벤트가 발생하지 않으면 연결을 종료하는 세션타임아웃 설정필요

-sh,ksh,bash 사용시

#cat /etc/profile

TMOUT=600 (초단위)

export TMOUT

-csh 사용시

#cat /etc/csh.login 또는 cat /etc/csh.cshrc

set autologout=10 (분단위)

=> 세션 타임아웃을 600초 이내로 설정

* root 홈, 패스 디렉터리 권한 및 패스 설정
- root 계정의 PATH 환경 변수에 “.” 이 포함되어 있으면 root 계정으로 접속한 관리자가 의도하지 않은 현재 디렉터리에 위치하고 있는 명령어가 실행될 수 있음 -> PAH 마지막으로 이동시키거나 불필요할 경우 삭제 조치 필요

- shell 에 따라 참조되는 환경설정파일이 다름
- bin/sh : /etc/profile, $HOME/.profile
- bin/bash : /etc/profile, $HOME/.bash_profile
- bin/ksh : /etc/profile, $HOME/.profile, $HOME/kshrc
- bin/csh : /etc/.login, $HOME/.cshrc, $HOME/,login
* 파일 및 디렉터리 소유자/소유그룹 설정
- 소유자/소유그룹이 존재하지 않는 디렉터리는 시스템 자원낭비가 될 수 있고 중요파일일 경우 관리가 되지 않는 문제점을 야기함

-> 실습을통해 사용자를 삭제하면 UID/GID 만 남아 있을 뿐 소유주/소유그룹이 표시되지 않는 것을 확인 할 수 있음
-> userdel -r algisa2

-> find .\(-nouser -o -nogroup \) -exec ls -al {} \;
: 소유자/소유그룹이 없는 파일과 디렉터리 검색

=> 파일 소유자/소유그룹을 변경하거나 불필요시 삭제하도록 한다
-> chown, chgrp

* world writable 파일 점검 (모든 사용자에게 쓰기 권한이 부여된 파일 점검)
- 모든 사용자가 접근 및 수정할 수 있는 권한으로 설정된 파일이 존재하면 일반 사용자의 실수 또는 악의적으로 주요 파일 정보를 변경할 수 있으므로 시스템 장애나 추가적인 공격에 활용될 수 있는 문제점이 있다

-> find -perm -2 -exec ls -al {} \;

-> 위 명령어를 통해 others 에 쓰기권한이 부여된 파일/디렉터리를 검색할 수 있음 (여기서 -2는 -0002 와동일함)

=> other (모든사용자) 에 대해 쓰기 권한을 삭제하거나 불필요한 파일/디렉터리일 경우 삭제 조치

* 주요 파일 소유자 및 권한 설정
- /etc/passwd : 사용자 정보를 담고 있는 파일 . root 소유의 644 이하의 권한을 설정
- /etc/shadow : 사용자의 암호화된 패스워드를 담고 있는 파일. root 소유의 400 이하의 권한을 설정
- /etc/hosts : IP와 호스트의 이름을 매핑하는데 사용되는 파일로 외부의 불법적인 접근에 의한 조작시 악의적인 시스템에 접근할 가능성이 있으므로 root 소유의 600 이하의 권한을 설정 (금융위원회 기준으로는 644)
- /etc/(x)inetd.conf 파일 : inetd 데몬에 의한 설정 파일로 외부의 불법적인 접근에 의해 악의적인 프로그램을 등록하고 root 권한으로 실행할 가능성이 있으므로 root 소유의 600 이하의 권한을 설정한다
- /etc/syslog.conf : syslogd 데몬에 대한 설정파일로 주요 로그파일에 대한 설정 정보를 가지고 있음. 외부의 침입에 의해 변경되는 경우 로그가 정상적으로 기록되지 않아 침입자의 흔적이나 시스템 오류사항을 정확히 분석할 수 없으므로 root 소유의 644 이하의 권한을 설정함
- /etc/services : 서비스 관리 (서비스별포트와 프로토콜) 정보를 담고 있는 파일로 불법적인 접근에 의해 변경되는 경우 정상적인 서비스를 제한하거나 허용되지 않는 서비스를 등록하는 등의 위험이 있으므로 root 644 권한을 설정

*** 윈도우 서버 취약점

* 윈도우 서버에서 Administrator 의 계정 정보를 변경 하여 취약점을 예방하도록 하자
* Guest 계정은 소프트웨어나 하드웨어설치, 설정변경, 암호만들기 등을할 수없고 외부 사용자가 컴퓨터를 잠시 접근할 수 있는 계정

으로, 불특정 다수의 접근이 필요한 경우 게스트 계정보다는 일반 사용자 계정을 생성하여 쓰는 것을 권장

* 불필요한 계정은 삭제하도록 하자
* 관리자 그룹에는 최소한의 계정만을 포함하며 관리자 그룹에 불필요한 계정이 있다면 해당 계정을 제거한다
* 패스워드 설정시 로컬 보안설정에서 암호 복잡성을 사용으로 바꾼다

-> 2가지 조합시 10자리 이상, 3가지 조합했을 때 8자리 이상 (영대문자/소문자/숫자/특수문자

-> 부적절한 패스워드 예시) 길이가 짧은 것, 사전에 나오는 단어나 단어들의 조합, 키보드 자판의 연속된 문자, 사용자 계정정보와 유사한 단어

* 최근 패스워드를 기억할 수 있도록 설정하여 최근 암호 기억을 12개 이상으로 설정하는 것을 권장한다

* 패스워드 최대 사용기간을 설정한다. 주기적으로 변경하는것이 중요하다

-> 90 일 이하 권장 (최대 사용기간)
-> 계정 속성의 '암호 사용기간 제한 없음'을 해제한다.

* 패스워드 최소 사용기간을 설정한다

-> 0보다 큰값으로 설정 (일반적으로 하루 설정)

* 패스워드 최소 길이 설정

-> 최소 암호길이8자이상 권장

* 해독 가능한 암호화 사용 설정

-> 반드시 필요한 경우를 제외하고는 '사용안함' 으로 설정함

* 계정 잠금 정책 -> 계정 잠금기간(60분이상, 무차별 공격 방지 가능), 계정잠금 임계값(로그인 횟수 제한. 5이하의 값 권장), 다음

시간 후 계정 잠금수를 원래대로 설정 (60분 이상, 로그인시도카운트가 0
으로 설정될 때 까지의 경과해야하는 시간)

* 하드 디스크 기본 공유 제거 : C$, D$ (기본공유) / ADMIN$, IPC$ (원격 관리 및 IPC용)

-> 기본공유를 사용하지 않으면 기본 공유 폴더를 통해 허가 받지 않은 사용자가 하드 디스크 내의 폴더나 파일에 접근하거나 바이러스가 침투하는 경로가 될 수 있음

-> net share 명령 또는 관리화면에서 우클릭을 통해 공유 중지가능
-> 재시작시 다시 생성 됨, 이럴 땐 레지스트리 수정 필요
-> 레지스트리 수정방법
(HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSoftSet\Services\LanmanServer\Parameters 를 선택한 후 'AutoShareServer' DWORD 값 생성 후 0 으로 설정

* 공유 권한 및 사용자 그룹 설정 -> 공유 폴더에 Everyone 그룹이 포함되어 있으면 익명 사용자에 대한 접근이 가능하므로 Everyone 그룹 제거 필요

* 불필요한 서비스제거 -> Alerter, Clipbook, Meessenger, Simpe TCP/ID services (Echo, Discard, Character Generator, Daytime, Quote of the Day)

* 로그 감사 정책 설정 -> 감사 정책이란 어떤 로그를 남길지 정의한 규칙을 의미함. 법적 요구 사항과 조직의 정책에 따라 필요한 로그를 남기도록 설정

- 감사안함 대상 : 개체 액세스, 시스템 이벤트, 프로세스 추적
- 실패에 대해서만 감사대상 : 계정관리, 권한사용, 디렉터리 서비스 액세스

- 모두 감사(실패와성공) : 계정 로그온 이벤트, 로그온 이벤트, 정책 변경
  -> 계정 로그온은 도메인 계정에 대한 로그온 성공/실패, 로그온은 로컬 계정에 대한 로그온 성공/실패 이벤트